ENGINEERING FOR AI SYSTEMS

AI Engineer Portfolio

AI를 모델 호출에서 끝내지 않고, 데이터·추론·API·인터페이스가 이어지는 서비스로 구현합니다.

핵심 포지셔닝

Python 기반 AI 시스템을 설계하고, 모델 처리 흐름을 백엔드 API와 사용자 인터페이스까지 연결합니다.

SELECTED PROJECTS

01 / Codebase AI Assistant

RAG로 Python 코드베이스를 검색하고, 관련 코드와 출처를 포함한 답변을 생성합니다.

Python · FastAPI · ChromaDB · Ollama

02 / AI Smart Surveillance Platform

YOLO11 탐지와 ByteTrack 추적으로 제한 구역 이벤트를 기록하고 웹 대시보드로 확인합니다.

YOLO11 · OpenCV · FastAPI · React

ENGINEERING CAPABILITIES

01 / AI PIPELINES

RAG 검색, 임베딩, 로컬 LLM 추론, 컴퓨터 비전과 객체 추적 흐름을 구현합니다.

02 / SERVICE INTEGRATION

FastAPI를 통해 AI 처리 결과를 API와 React·TypeScript 인터페이스에 연결합니다.

03 / DEBUG & ITERATE

구현 한계와 입력·상태 흐름을 확인하고, 다음 개선 항목을 구분합니다.

WORKING METHOD

문제 정의	시스템 설계	구현	검증	개선

CONTACT & LINKS

EMAIL	soundaim56@gmail.com
GITHUB	github.com/Hyuki56
PORTFOLIO	hyuki56.github.io/ai-engineer-portfolio

PROJECT 01 / LLM · RAG

Codebase AI Assistant

코드 구조를 검색 가능한 근거로 바꾸는 로컬 RAG 서비스

github.com/Hyuki56/codebase-ai-assistant →

문제 코드가 여러 파일과 함수에 나뉘어 있으면 동작의 위치와 맥락을 찾기 어렵습니다. 저장소 전체를 LLM에 전달하는 방식은 입력 문맥과 근거 추적에 제약이 있습니다.

RAG PIPELINE · CODE LOCATIONS

01 / 수집·청킹	02 / 임베딩·저장	03 / 검색·문맥	04 / 답변·API
Python 파일 로딩·AST 분할 loader/github_loader.py chunker/code_chunker.py	Ollama 임베딩·ChromaDB embedding/embedder.py vectordb/chroma_manager.py	관련 코드·경로 검색 services/codebase_service.py	로컬 LLM·FastAPI llm/ollama_client.py · api/api.py

ENGINEERING

색인·질의 API

POST /api/index에서 저장소를 색인하고 POST /api/ask에서 질문 임베딩, 관련 코드 검색, 문맥 구성과 답변 생성을 수행합니다.

검색 근거와 출처 반환

검색된 코드의 파일 경로를 메타데이터로 관리하고 답변과 함께 반환해 관련 코드를 직접 확인할 수 있도록 했습니다.

로컬 모델 구성

nomic-embed-text와 qwen2.5-coder:7b를 Ollama에서 사용하고 ChromaDB를 벡터 저장소로 활용합니다.

AST 기반 코드 단위

모듈 최상위 함수와 클래스를 AST로 추출합니다. 긴 단위는 줄바꿈 기준으로 나누며 파싱 실패 파일은 텍스트 분할로 처리합니다.

PROBLEM SOLVING · PROBLEM → RESPONSE → OUTCOME

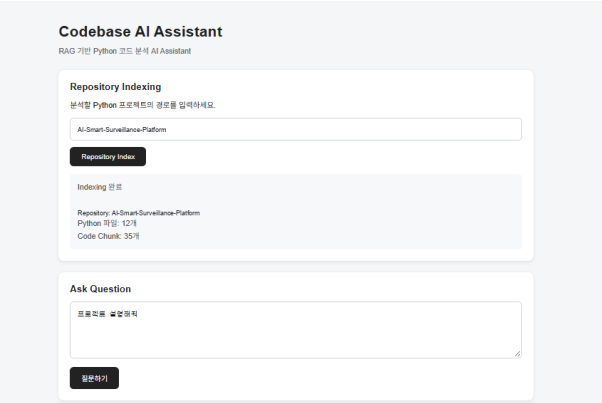
01 / 코드 구조와 입력 길이

문제 줄 수 기준으로만 나누면 함수·클래스가 끊겨 맥락이 손실되고, 긴 코드 단위는 모델 입력 한도를 넘을 수 있습니다.
대응 AST로 모듈 최상위 함수·클래스를 나누고 1,500자를 넘는 Chunk는 줄바꿈 기준으로 추가 분할했습니다. 파싱 실패 파일은 텍스트 기반으로 처리했습니다.
결과 구조를 고려한 코드 검색 단위를 만들었습니다. 검색 품질의 정량 개선 수치는 측정하지 않았습니다.

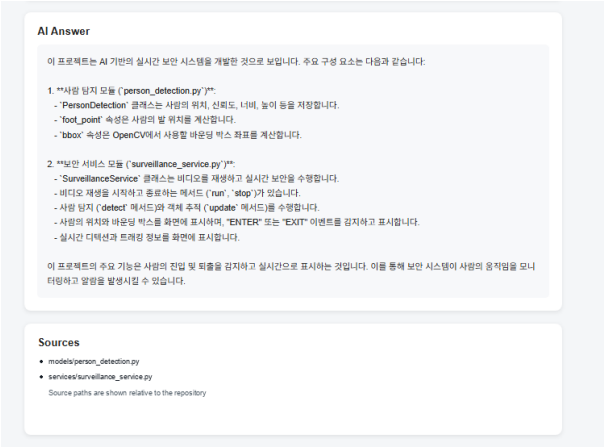
02 / 답변 근거 추적

문제 답변만 보면 어떤 코드가 검색되어 생성 근거로 쓰였는지 확인하기 어렵습니다.
대응 검색된 Chunk의 파일 경로를 메타데이터로 관리하고 답변과 함께 출처 경로를 반환했습니다.
결과 사용자가 답변에서 관련 파일을 찾아 직접 확인할 수 있습니다.

INTERFACE EVIDENCE · DEMO INDEXED THE PJ2 REPOSITORY



색인 화면 · AI-Smart-Surveillance-Platform
Python 12개 · Chunk 35개



질의응답 화면 · 답변과 검색 출처 경로

참고: PJ1 자체 저장소가 아니라 PJ2 감시 프로젝트 저장소를 색인한 교차 저장소 질의 사례입니다.

현재 한계 전체 재색인 방식이며 코드 요소 관계 그래프와 검색·답변 품질의 정량 평가는 없습니다.
다음 단계 검색 평가, 증분 색인, 검색 결과 재정렬과 출처 표시를 개선합니다.

PROJECT 02 / COMPUTER VISION · BACKEND API · WEB DASHBOARD

AI Smart Surveillance Platform

파일 영상 입력을 탐지·추적해 제한 구역 이벤트로 기록하는 시스템

github.com/Hyuki56/AI-Smart-Surveillance-Platform →

문제 사람 탐지만으로는 제한 구역 진입 여부와 발생 상황을 확인하기 어렵습니다. 탐지를 추적·구역 판정에 연결하고 이벤트를 기록해 웹 화면에서 확인하도록 구성했습니다.

DETECTION-TO-EVENT PIPELINE

01 / 사람 탐지	02 / 객체 추적	03 / 구역 판정	04 / 이벤트·화면
YOLO11 · 중복 박스 정리	ByteTrack · ID 연결	Foot Point · Polygon	ENTER/EXIT · SQLite · API · React

ENGINEERING

탐지 결과와 객체 추적

YOLO11 사람 탐지 결과에서 IoU·포함률 기준으로 중복 박스를 정리한 뒤 ByteTrack으로 전달해 프레임 간 ID를 추적합니다.

발 위치 기반 구역 판정

바운딩 박스 하단 중앙을 Foot Point로 사용하고 OpenCV pointPolygonTest로 고정 Polygon 안팎을 판정합니다.

상태 전이 이벤트 처리

OUTSIDE → INSIDE에서 ENTER, 반대 전환에서 EXIT를 생성합니다. 진입 때 시각·스냅샷·DB 레코드를 저장하고 이탈 때 체류 시간을 갱신합니다.

스트림과 대시보드 연동

FastAPI의 시작·중지·상태·이벤트·스냅샷 API와 MJPEG 프레임 스트림을 React·TypeScript 화면에 연결합니다.

PROBLEM SOLVING · PROBLEM → RESPONSE → OUTCOME

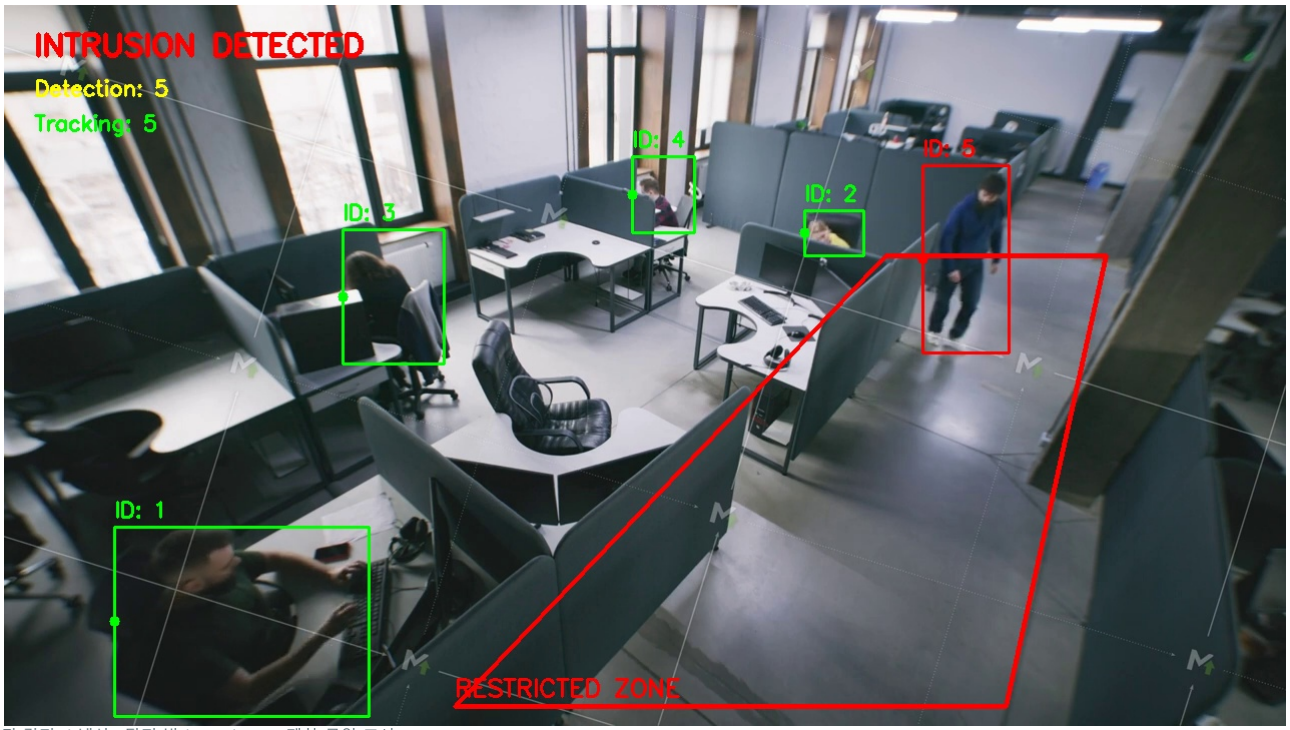
01 / 구역 경계 판정

문제 바운딩 박스 중심은 실제 발이 닿는 바닥 접점과 달라 구역 경계 판정에 적합하지 않을 수 있습니다.
대응 박스 하단 중앙을 Foot Point로 정의하고 해당 좌표를 Polygon 내부 판정에 사용했습니다.
결과 사람의 바닥 접점을 기준으로 구역 상태를 계산하고 ENTER·EXIT 이벤트에 연결했습니다.

02 / 이벤트 증거 확보

문제 DB 이벤트 정보만으로는 발생 당시 영상 속 탐지 상황을 확인하기 어렵습니다.
대응 탐지 박스·추적 ID·구역 표시가 그려진 프레임을 ENTER 시점에 저장합니다. EXIT에서는 체류 시간을 DB에 갱신합니다.
결과 이벤트 기록에서 진입 스냅샷과 이탈 정보를 확인할 수 있습니다.

ACTUAL DETECTION SNAPSHOT · LOCAL VIDEO FILE DEMO



저장된 감지 스냅샷 · 탐지 박스, tracker ID, 제한 구역 표시

현재 한계: 영상 경로와 구역 좌표는 코드에 고정되어 있습니다. 실시간 CCTV 입력, UI 구역 지정, 정량 평가와 장시간 안정성 검증은 향후 과제입니다.
개선 계획: CCTV 입력과 마우스 기반 구역 설정, 탐지·이벤트 평가, 추적 상태 만료와 API 접근 제어를 추가합니다.
TECH STACK Python · YOLO11 · OpenCV · ByteTrack · FastAPI · React · TypeScript · SQLite

EVIDENCE AND SYSTEM FLOWS

실행 근거와 데이터 흐름

두 프로젝트의 입력부터 결과 확인까지, 구현 경로와 검증 범위를 비교합니다.

01 / CODEBASE AI ASSISTANT · RAG

01 / SOURCE	02 / CHUNK	03 / INDEX	04 / RETRIEVE
Python 저장소 파일 경로·코드	AST 최상위 함수·클래스 약 1,500자 기준 추가 분할	Ollama 임베딩 nomic-embed-text	ChromaDB 검색 코드·경로 메타데이터

질문 임베딩 → 관련 코드 검색 → qwen2.5-coder:7b 답변 생성 → 출처 파일 경로 반환

API: POST /api/index · POST /api/ask

02 / AI SMART SURVEILLANCE · FILE VIDEO TO EVENT

01 / INPUT	02 / DETECT · TRACK	03 / ZONE · EVENT	04 / SERVICE
videos/office.mp4 파일 영상	YOLO11 사람 탐지 ByteTrack ID 추적	Foot Point · Polygon ENTER / EXIT	SQLite · FastAPI MJPEG · React UI

파일 영상 프레임 처리 → 탐지 및 ID 연결 → 구역 상태 전이 → 이벤트·스냅샷 기록 → API와 대시보드 표시

API: POST /start · POST /stop · GET /status · GET /events · GET /video-feed

EVIDENCE AND VERIFICATION SCOPE

자료	확인 가능한 내용	범위
PJ1 색인 화면	대상 저장소, 파일·Chunk 수	PJ2 저장소 색인 데모
PJ1 답변 화면	답변과 출처 파일 경로	교차 저장소 질의
PJ2 감지 스냅샷	탐지 박스, ID, 구역 표시	제공된 저장 이미지

검증 범위

소스 구조와 제공된 화면 자료를 바탕으로 작성했습니다. 검색 적합도·탐지 정확도·FPS·장시간 안정성은 정량 검증하지 않았으며, 공개 운영 환경에 배포 완료된 서비스가 아닙니다.

PJ1 GitHub · PJ2 GitHub · Web Portfolio